

M6_K_Nearest_Neighbors_KNN

January 23, 2026

0.0.1 K-Nearest Neighbors (KNN)

KNN és un algorisme que classifica mostres noves basant-se en les K mostres més properes (veïnes) de les dades d'entrenament. És així de simple i intuïtiu!

Tot i que és molt utilitzat en problemes de classificació també és utilitzat en problemes de regressió. No ho haviem vist fins ara però existeixen els models híbrids determinats pels tipus de dades: categories (classificació) o numèrics (regressió).

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report, accuracy_score
```

Veguem primer una petita explicació visual:

```
[ ]: n_points = 50 #generem 50 punts "aleatoris"

# Tres grups (classes) amb centres diferents
X0 = np.random.randn(n_points//3, 2) + np.array([2, 2])
X1 = np.random.randn(n_points//3, 2) + np.array([6, 6])
X2 = np.random.randn(n_points//3, 2) + np.array([8, 2])

X = np.vstack((X0, X1, X2))
y = np.array(
    [0]*(n_points//3) +
    [1]*(n_points//3) +
    [2]*(n_points//3)
)

#Punt nou a classificar
new_point = np.array([[5, 4]])
```

```

# Valor de K (prova de modificar-lo i veure que passa)
K = 3 # PROVA: 3, 5, 7, 11...

knn = KNeighborsClassifier(n_neighbors=K)
knn.fit(X, y)

distances, indices = knn.kneighbors(new_point)
neighbors = X[indices[0]]
neighbor_classes = y[indices[0]]
prediction = knn.predict(new_point)

plt.figure(figsize=(10, 8))

colors = ['red', 'blue', 'green']
labels = ['Classe 0', 'Classe 1', 'Classe 2']

# Dades d'entrenament generades de forma "random"
for i in range(3):
    mask = y == i
    plt.scatter(X[mask, 0], X[mask, 1],
                c=colors[i], s=80, alpha=0.6,
                label=labels[i])

# Punt nou
plt.scatter(new_point[0, 0], new_point[0, 1],
            c='yellow', s=300, marker='*',
            edgecolors='black', linewidth=2,
            label='Punt nou', zorder=5)

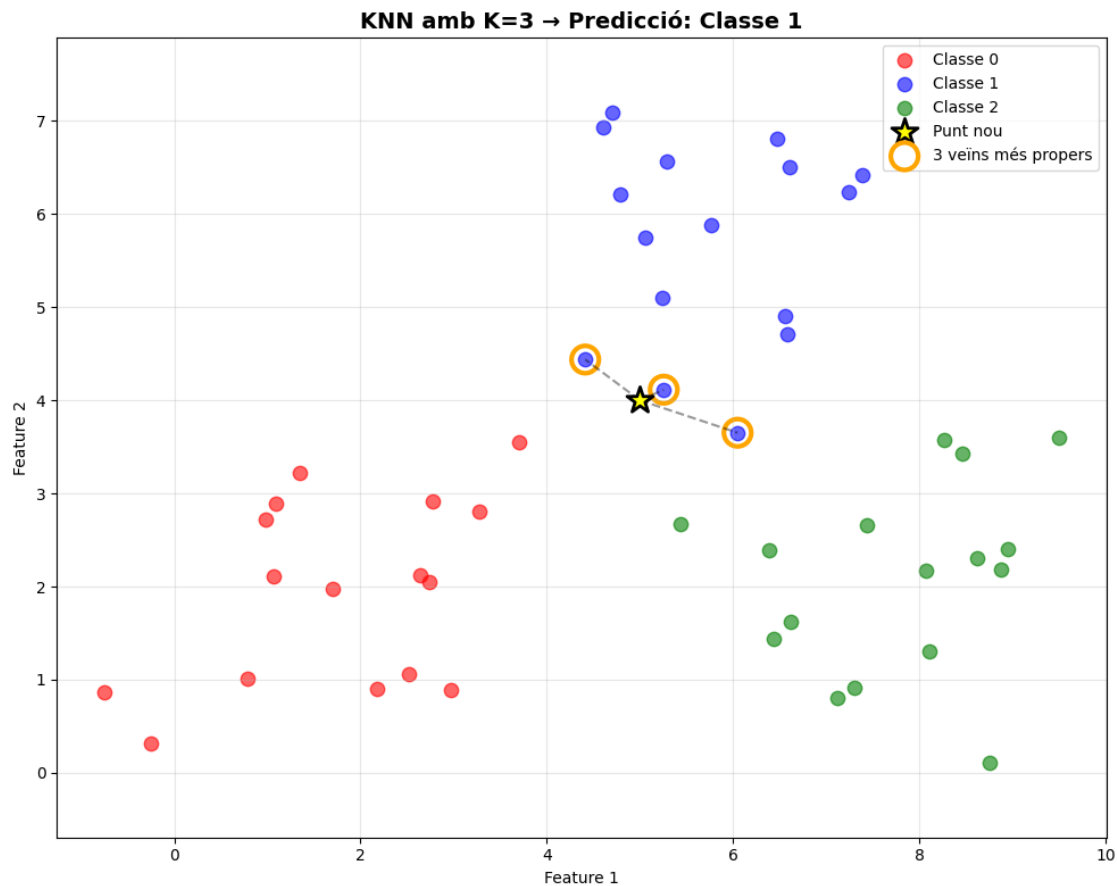
# KNN (Veïns més propers)
plt.scatter(neighbors[:, 0], neighbors[:, 1],
            s=300, facecolors='none',
            edgecolors='orange', linewidth=3,
            label=f'{K} veïns més propers')

# Línies amb les Distàncies
for neighbor in neighbors:
    plt.plot([new_point[0, 0], neighbor[0]],
             [new_point[0, 1], neighbor[1]],
             'k--', alpha=0.4)

plt.title(f'KNN amb K={K} → Predicció: Classe {prediction[0]}',
          fontsize=14, fontweight='bold')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(alpha=0.3)

```

```
plt.axis('equal')
plt.tight_layout()
plt.show()
```



Veiem ara com funciona aquest model amb les dades del càncer de mama, que ja hem vist anteriorment.

```
[ ]: # Carreguem les dades
breast_cancer = load_breast_cancer()
df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
df['target'] = breast_cancer.target

X = df.drop('target', axis=1)
y = df['target'].values
```

```
[ ]: # Dividim de les dades en train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
) #stratify=y perquè les dades de train i test quedin balancejades
```

```
[ ]: knn_basic = KNeighborsClassifier(n_neighbors=5) # K = 5
knn_basic.fit(X_train, y_train)

# Prediccions
y_pred_basic = knn_basic.predict(X_test)

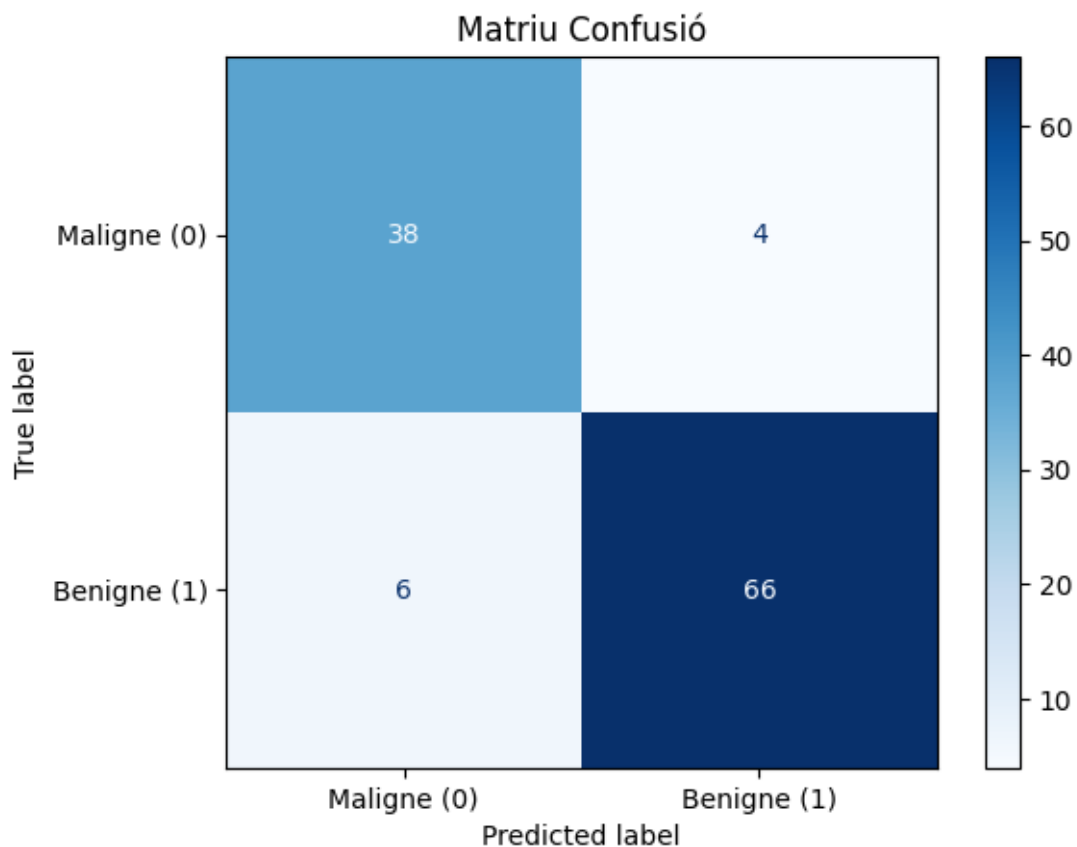
# Mètriques
acc_train_basic = knn_basic.score(X_train, y_train)
acc_test_basic = knn_basic.score(X_test, y_test)

print(f"    Accuracy Train: {acc_train_basic:.4f}")
print(f"    Accuracy Test:   {acc_test_basic:.4f}")
print(f"    Diferència:       {abs(acc_train_basic - acc_test_basic):.4f}")

# Matriu de confusió
cm = confusion_matrix(y_test, y_pred_basic)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Maligne_
↪(0)', 'Benigne (1)']) # Ordre real: [positiu, negatiu]
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()

# Report numèric
print(f"\nReport de Classificació:")
print(classification_report(y_test, y_pred_basic, target_names=breast_cancer.
↪target_names))
```

```
Accuracy Train: 0.9473
Accuracy Test:  0.9123
Diferència:     0.0350
```



Report de Classificació:

	precision	recall	f1-score	support
malignant	0.86	0.90	0.88	42
benign	0.94	0.92	0.93	72
accuracy			0.91	114
macro avg	0.90	0.91	0.91	114
weighted avg	0.91	0.91	0.91	114

Recordem que és una bona pràctica escalar les dades. Veguem com varien els resultats, ja que el model KNN és molt sensible a l'escalat.

```
[ ]: #Escalar/Normalitzar
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[ ]: knn_basic = KNeighborsClassifier(n_neighbors=5) #  $K = 5$ 
knn_basic.fit(X_train_scaled, y_train)

# Prediccions
y_pred_scaler = knn_basic.predict(X_test_scaled)

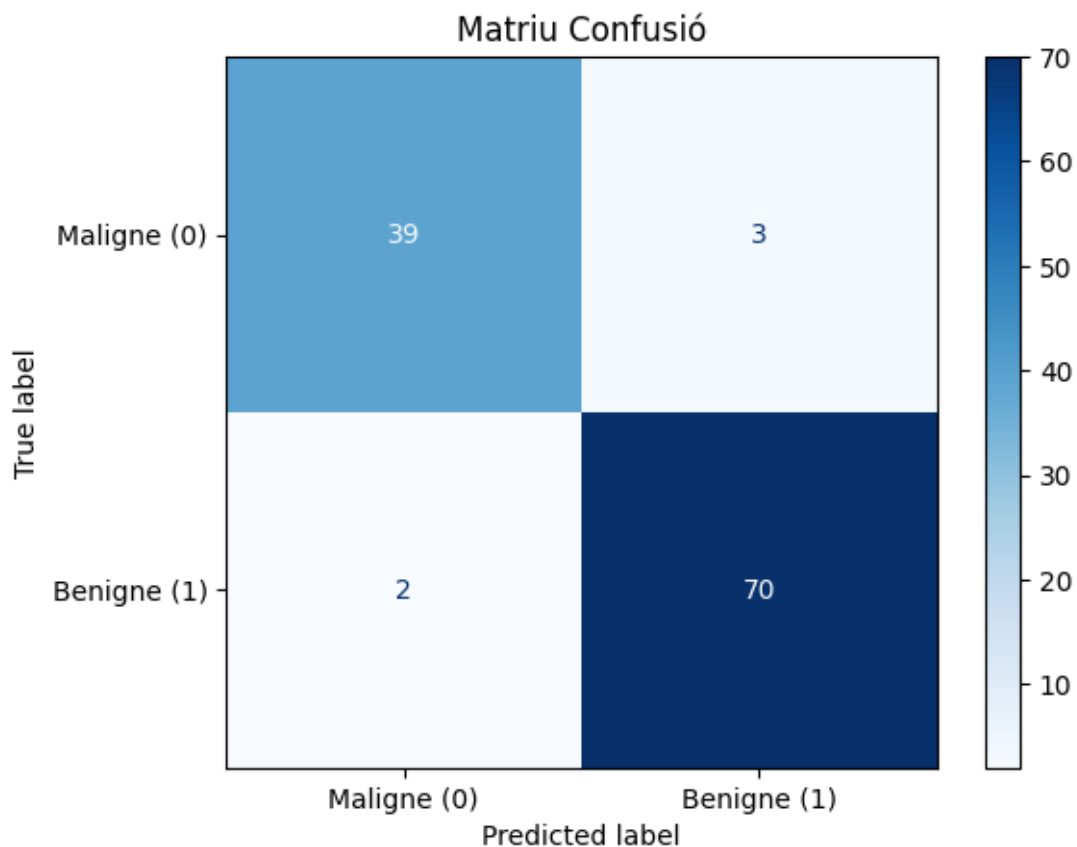
# Mètriques
acc_train_basic = knn_basic.score(X_train_scaled, y_train)
acc_test_basic = knn_basic.score(X_test_scaled, y_test)

print(f"    Accuracy Train: {acc_train_basic:.4f}")
print(f"    Accuracy Test:   {acc_test_basic:.4f}")
print(f"    Diferència:       {abs(acc_train_basic - acc_test_basic):.4f}")

# Matriu de confusió
cm = confusion_matrix(y_test, y_pred_scaler)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Maligne_
↪(0)', 'Benigne (1)']) # Ordre real: [positiu, negatiu]
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()

# Report numèric
print(f"\nReport de Classificació:")
print(classification_report(y_test, y_pred_scaler, target_names=breast_cancer.
↪target_names))
```

```
Accuracy Train: 0.9736
Accuracy Test:  0.9561
Diferència:     0.0175
```



Report de Classificació:

	precision	recall	f1-score	support
malignant	0.95	0.93	0.94	42
benign	0.96	0.97	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

Veiem com la normalització/estandarització millora el model significativament!

Veiem ara com obtenir el millor valor de K amb *CROSSVALIDATION* (provar diferents valors i comprovar-ne el resultat). Ens ajudarà a evitar l'overfitting i trobar el valor òptim.

```
[ ]: # Provar diferents K
k_values = range(1, 31)
cv_scores_mean = []
cv_scores_std = []
```

```

train_scores_list = []

for k in k_values:
    knn_temp = KNeighborsClassifier(n_neighbors=k)

    # Cross-validation
    cv_scores = cross_val_score(knn_temp, X_train_scaled, y_train, cv=5)
    cv_scores_mean.append(cv_scores.mean())
    cv_scores_std.append(cv_scores.std())

    # Train score
    knn_temp.fit(X_train_scaled, y_train)
    train_scores_list.append(knn_temp.score(X_train_scaled, y_train))

# Trobar millor K
best_k = k_values[np.argmax(cv_scores_mean)]
best_cv_score = max(cv_scores_mean)

print(f"\nMILLOR K TROBAT: {best_k}")
print(f"CV Score: {best_cv_score:.4f}")

```

MILLOR K TROBAT: 8

CV Score: 0.9714

Fem la comprovació i càlculs per a K=8

```

[ ]: knn_basic = KNeighborsClassifier(n_neighbors=8)
knn_basic.fit(X_train_scaled, y_train)

# Prediccions
y_pred_scaler = knn_basic.predict(X_test_scaled)

# Mètriques
acc_train_basic = knn_basic.score(X_train_scaled, y_train)
acc_test_basic = knn_basic.score(X_test_scaled, y_test)

print(f"    Accuracy Train: {acc_train_basic:.4f}")
print(f"    Accuracy Test:   {acc_test_basic:.4f}")
print(f"    Diferència:      {abs(acc_train_basic - acc_test_basic):.4f}")

```

Accuracy Train: 0.9758

Accuracy Test: 0.9737

Diferència: 0.0021

Veiem la matriu de confisió obtinguda:

```

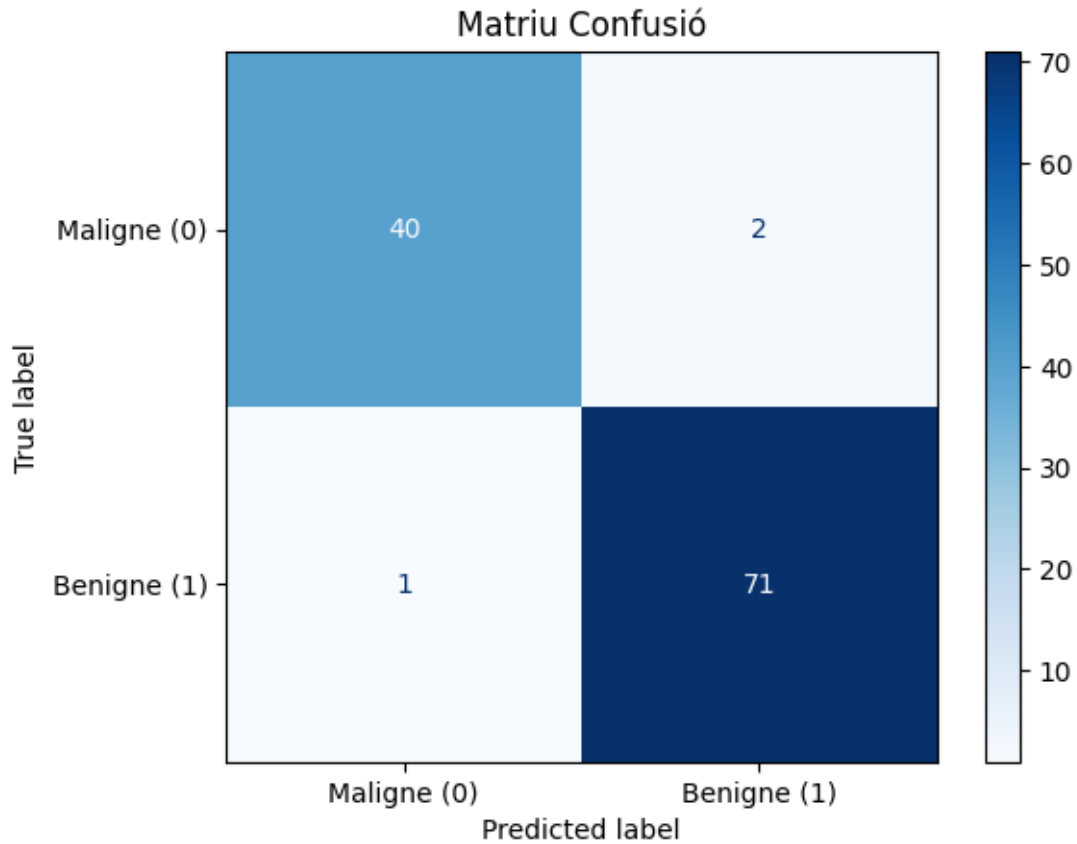
[ ]: cm = confusion_matrix(y_test, y_pred_scaler)

```

```

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Maligne_
↪(0)', 'Benigne (1)']) # Ordre real: [positiu, negatiu]
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()

```



Hi ha més paràmetres del model que podem “ajustar” (avançat) i el que se sol fer es provar totes les opcions i trobar l’òptim:

- **n_neighbors:** Nombre de veïns més propers que mira el KNN per votar la predicció.
- **weights:** Com ponderen els vots dels veïns. ‘uniform’ dona el mateix pes a tots els veïns (democràcia total) o ‘distance’ dona més pes als veïns més propers.
- **metric:** Tipus de distància per mesurar “com de semblants són” dos punts: ‘euclidean’ (distància recta); manhattan (distància en bloc), ‘minkowski’ (generalitza ambdues).

p: Paràmetre de la distància Minkowski. $p=1$ = Manhattan (suma diferències absolutes); $p=2$ = Euclidiana (arrel quadrada suma quadrats).

```

[ ]: # Definir grid de paràmetres
param_grid = {

```

```

    'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski'],
    'p': [1, 2] # Només per Minkowski
}

# Grid Search
knn_grid = KNeighborsClassifier()
grid_search = GridSearchCV(
    knn_grid,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1,
    return_train_score=True
)

```

```

[ ]: import time
start_time = time.time()
grid_search.fit(X_train_scaled, y_train)
elapsed_time = time.time() - start_time

print(f"\nGrid Search completat en {elapsed_time:.2f} segons!")

```

Fitting 5 folds for each of 84 candidates, totalling 420 fits

Grid Search completat en 14.46 segons!

```

[ ]: print(f"\nMILLORS PARÀMETRES:")
for param, value in grid_search.best_params_.items():
    print(f"    {param}: {value}")

```

```

MILLORS PARÀMETRES:
  metric: euclidean
  n_neighbors: 3
  p: 1
  weights: uniform

```

```

[ ]: # Model final amb millors paràmetres
knn_final = grid_search.best_estimator_

y_pred_final = knn_final.predict(X_test_scaled)
y_train_pred_final = knn_final.predict(X_train_scaled)
acc_train_final = accuracy_score(y_train, y_train_pred_final)
acc_test_final = accuracy_score(y_test, y_pred_final)

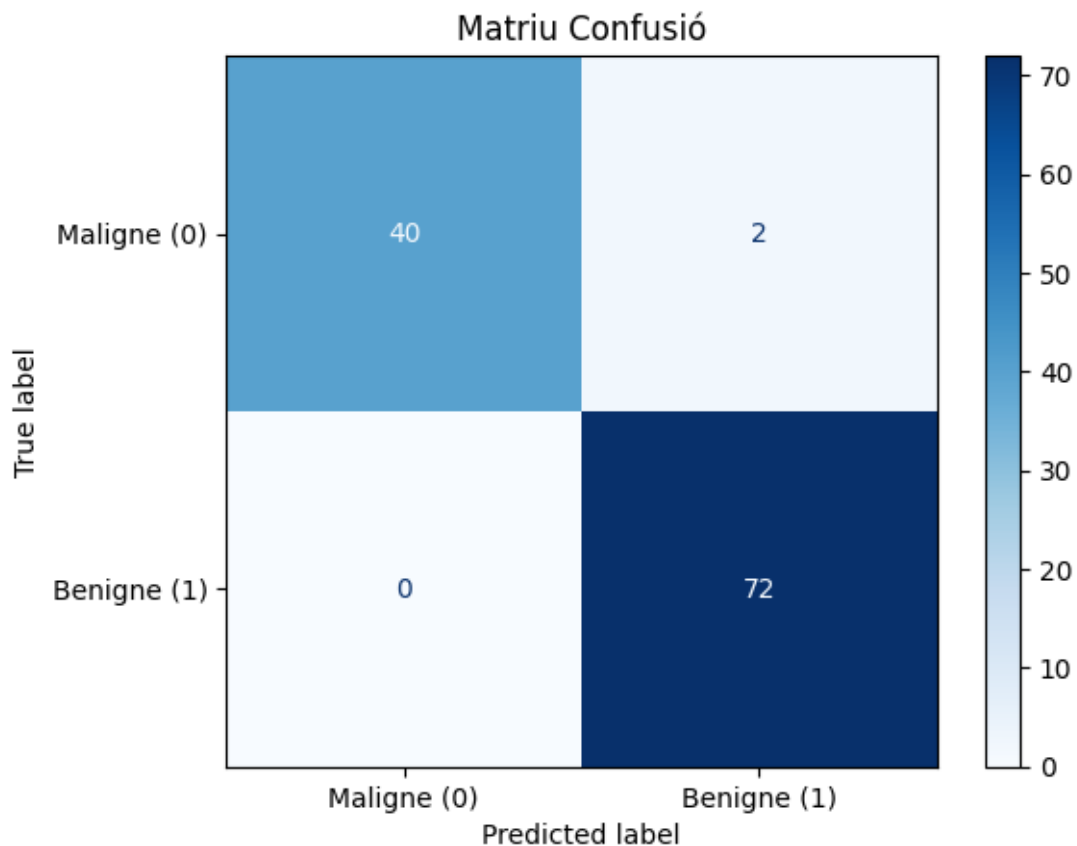
```

```
diferencia_final = abs(acc_train_final - acc_test_final)

print(f"    Accuracy Train Final: {acc_train_final:.4f}")
print(f"    Accuracy Test Final:   {acc_test_final:.4f}")
print(f"    Diferència Final:       {diferencia_final:.4f}")
```

```
Accuracy Train Final: 0.9780
Accuracy Test Final:  0.9825
Diferència Final:    0.0044
```

```
[ ]: cm = confusion_matrix(y_test, y_pred_final)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Maligne_
↳(0)', 'Benigne (1)']) # Ordre real: [positiu, negatiu]
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()
```



0.0.2 Fine-tuning, Cross-Validation i GridSearchCV

Fine-tuning: significa optimitzar els paràmetres del model (com `n_neighbors`, `max_depth`, `C`, etc.) per obtenir el millor rendiment possible. En lloc de triar `n_neighbors=5`. El *fine-tuning* prova sistemàticament totes les combinacions per trobar la configuració òptima.

Cross-Validation (CV): és una validació més fiable. Quan, per exemple fem `cv=5`, dividim les dades d'entrenament en 5 parts i entrenes/valides 5 vegades:

Fold 1: Entrena(80%) + Valida(20%) → accuracy=87%

Fold 2: Entrena(80%) + Valida(20%) → accuracy=89%

Fold 3: Entrena(80%) + Valida(20%) → accuracy=85%

Fold 4: Entrena(80%) + Valida(20%) → accuracy=88%

Fold 5: Entrena(80%) + Valida(20%) → accuracy=90%

MITJANA CV: 87.8% ← Molt més fiable que una sola divisió!

La CV mitjana dona el rendiment real esperat, perquè de vegades els bons o mals resultats poden ser casualitat.

GridSearchCV: Combina el *Fine-tuning* i el *Cross-Validation*